2026학년도 2학기 인문데이터 수량분석

제15강 이미지 데이터

박수민

서울대학교 인문대학 연계전공 인문데이터과학

2026년 11월 9일 월요일

오늘의 목표

1. 사진이 수의 배열임을 설명하고, 사진을 픽셀 하나가 한 행인 표로 펼칠 수 있다.
2. RGB와 HSV의 차이를 설명하고, 밝기·채도·색상으로 사진 수천 장을 훑을 수 있다.
3. 객체·얼굴·자세 탐지 결과 표를 읽어 사진 위에 겹치고, 그 결과를 의심할 수 있다.
4. 임베딩으로 사진의 지도를 그리고 비슷한 사진을 찾을 수 있다.

사전 준비

새로 설치할 패키지는 없다. 8강에서 받은 `ggimg`, `hdir`를 쓴다 (`jpeg`는 `ggimg`와 함께 설치되었다).

```
library(tidyverse)
library(ggimg)
library(hdir)
library(jpeg)
```

1 **사진을 표로**
컴퓨터가 보는 사진
표로 펴기

2 색: RGB에서 HSV로

3 사물 찾기: 컴퓨터 비전

4 사람 찾기: 얼굴과 자세

5 비슷한 사진 찾기: 임베딩

6 마무리

7강과 13강이 남긴 숙제

미뤄 둔 말

7강: “색이 있는 사진은 빨강·초록·파랑 세 장의 행렬을 겹쳐 놓은 것이다. 15강에서 다룬다.”
13강: “타일은 픽셀로 된 래스터, 곧 그림이다.”

그림 자체가 자료다

미국 의회도서관, 게티 미술관, 유로피아나, 레이크스 미술관, 국립중앙박물관 e뮤지엄… 디지털로 옮긴 그림과 사진은 이미 사람이 다 볼 수 없을 만큼 많다. 교재의 저자들은 이를 컴퓨터로 멀리서 보는 방법을 **distant viewing**이라 부른다.

- ➔ 8강의 “읽지 않고 읽기”를 그림에 옮긴 셈이다.
- 💡 글자를 찍은 스캔본을 글자로 되돌리는 일(OCR)은 오늘 다루지 않는다. 오늘은 **보이는 것**을 다룬다.

오늘의 자료: FSA-OWI 컬러 사진



미국 정부의 기록 사진

농업안정국(FSA)과 전시정보국(OWI)의 사진단이 찍었다. 흑백 약 17만 장 곁에 컬러가 약 1,600장 있다.

- FSA는 주로 농촌과 농업 노동을, OWI는 공장·철도·항공 훈련 등 전시 동원을 찍었다 (미국 의회도서관 안내).

→ 대공황의 흑백 사진으로 유명한 컬렉션의, 잘 알려지지 않은 컬러 쪽이다. 오늘 12장을 자주 본다.

메타데이터: 표 한 줄이 파일 한 개

```
fsac <- read_csv(
  file.path("hdr", "data", "fsac_metadata.csv.bz2")
)
fsac |> select(filename, year, title, height, width)
```

```
# A tibble: 1,615 × 5
    filename  year title                                              height width
       <dbl> <dbl> <chr>                                               <dbl> <dbl>
1 2017877351  1939 Commuters, who have just come off the train…        715  1024
2 2017877359  1939 Railroad cars and factory buildings in Lawr…       715  1024
3 2017877451  1939 Copper mining and sulfuric acid plant, Copp…       728  1024
# i 1,612 more rows
```

- 지금까지의 자료와 달리 내용은 표 안에 없다. 사진은 **파일 한 개에 한 장씩** 따로 있고, 표는 그 파일을 가리킨다.

➔ `filename`이 키다(5강). 뒤에 나올 탐지 결과 표를 이 열로 잇는다.

경로부터 고쳐 둔다

```
gimg_load_img(fsac$thm_path[1])
```

```
Error in jpeg::readJPEG(path) :
  unable to open data/fsac/thm/2017877351.jpg
```

```
fsac <- fsac |>
  mutate(
    thm_path = file.path("hdr", thm_path),
    med_path = file.path("hdr", med_path)
  )
```

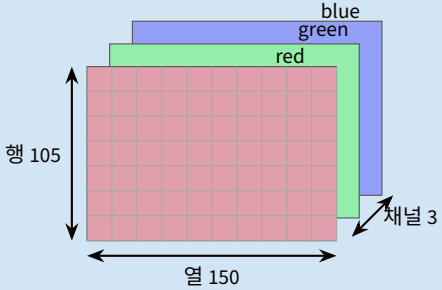
⚠ 표의 경로는 hdr 폴더 안에서 본 경로다. 우리 작업 디렉토리는 그 바깥이므로 앞에 "hdr"를 붙인다.

- thm_path는 폭 150픽셀 안팎의 섬네일로 1,615장 모두 있다. med_path는 폭 1,024픽셀의 큰 사진인데 받은 자료에는 **두 장만** 들어 있다.

gimg_load_img(): 사진을 수로

```
img <- gimg_load_img(
  fsac$thm_path[1]
)
dim(img)
```

```
[1] 105 150   3
```



행, 열, 채널

세로 105칸, 가로 150칸인 격자가 석 장 겹쳐 있다. 이렇게 겹친 행렬을 **배열**(array)이라 한다.

➔ 7강의 화소(pixel) 행렬이 세 장이 되었다. 흑백은 한 장, 투명도를 담은 사진은 네 장이다.

수를 들여다보기

```
1 img[1:2, 1:3, ]
```

```
, , 1

          [,1]      [,2]      [,3]
[1,] 0.1215686 0.1411764 0.1450980
[2,] 0.1450980 0.0784313 0.1254902

, , 2

          [,1]      [,2]      [,3]
[1,] 0.2039216 0.2196078 0.2235294
[2,] 0.2235294 0.1568627 0.2000000

, , 3

          [,1]      [,2]      [,3]
[1,] 0.0000000 0.0117647 0.0313725
[2,] 0.0156862 0.0000000 0.0196078
```

읽는 법

0은 색상 채널의 빛을 끄고, 1은 가장 밝게 켠다. 7강의 0–255를 255로 나눈 값이다($31/255 = 0.1216$).

왼쪽 위의 픽셀

R 0.12, G 0.20, B 0.00

➔ 어두운 풀색 한 점. 사진은 이런 점이 15,750개 모인 것이다.

hdir_pixels_long(): 픽셀 하나가 한 행

```
pix <- hdir_pixels_long(fsac$thm_path[1], fsac$filename[1])
pix
```

```
# A tibble: 15,750 × 12
  filename    row   col  nrow  ncol   red green   blue   hue saturation value
     <dbl> <dbl> <dbl> <int> <int> <dbl> <dbl>  <dbl> <dbl>      <dbl> <dbl>
1   2.02e9     1     1   105   150 0.122 0.204 0      0.234      1     0.204
2   2.02e9     2     1   105   150 0.145 0.224 0.0157 0.230      0.930 0.224
3   2.02e9     3     1   105   150 0.141 0.220 0.0275 0.235      0.875 0.220
# i 15,747 more rows
# i 1 more variable: hex <chr>
```

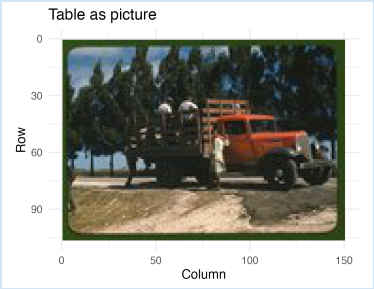
- 배열을 5강의 긴 형식으로 편 것이다. $105 \times 150 = 15{,}750$행.

➔ 표가 되었으니 새로 배울 것이 없다. `filter`, `group_by`, `ggplot`을 그대로 쓴다.

- hue, `saturation`, `value`는 잠시 뒤에, `hex`는 바로 다음에 쓴다. `2.02e9`는 긴 번호를 줄여 보인 것이다.

표를 다시 그림으로

```
truck <- hdir_pixels_long(
  fsac$thm_path[8], fsac$filename[8]
)
truck |>
  ggplot(aes(col, row)) +
    geom_raster(aes(fill = hex)) +
    scale_fill_identity() +
    scale_y_reverse() +
    coord_equal() +
    labs(
      title = "Table as picture",
      x = "Column", y = "Row"
    )
```



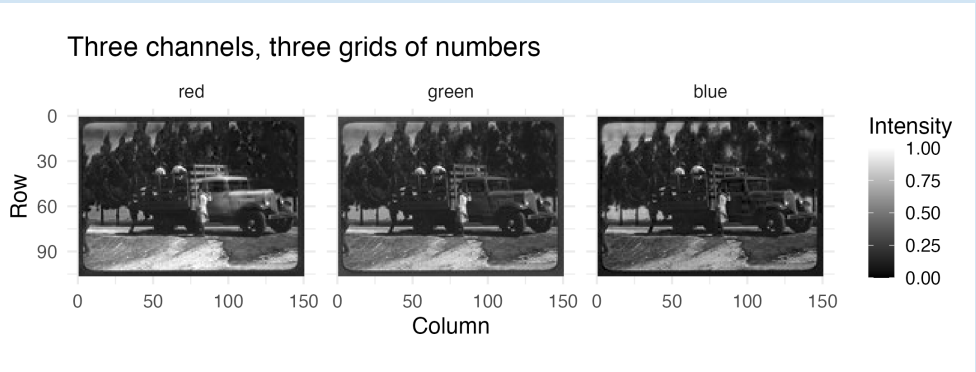
➔ 사진 8번이 3강의 도구로 되살아난다. hex가 곧 색이라 그대로 칠한다.

⚠ 행 번호는 **위에서 아래로** 늘어난다. 그래서 y축을 뒤집는다.

채널을 나누어 보면

```
truck |>
  pivot_longer(
    c(red, green, blue),
    names_to = "channel", values_to = "intensity"
  ) |>
  mutate(channel = fct_inorder(channel)) |>
  ggplot(aes(col, row)) +
    geom_raster(aes(fill = intensity)) +
    scale_fill_gradient(low = "black", high = "white") +
    scale_y_reverse() +
    coord_equal() +
    facet_wrap(~channel) +
    labs(
      title = "Three channels, three grids of numbers",
      x = "Column", y = "Row", fill = "Intensity"
    )
```

세 장의 흑백 사진



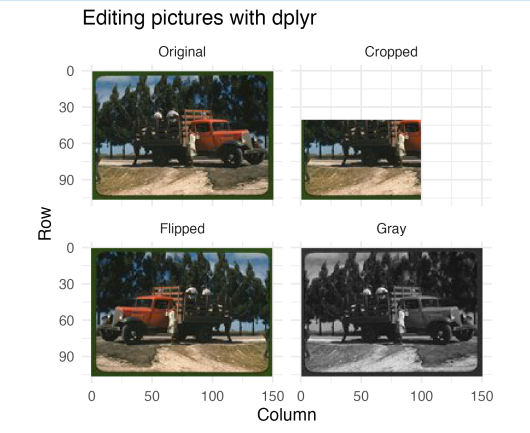
- 트럭의 차체는 red에서 밝고 blue에서 어둡다. 빨간 빛을 많이 켠 픽셀이다.
- 왼쪽 위 하늘은 blue에서 가장 밝다. 아래쪽 흙길은 세 채널 모두 밝다.

dplyr로 사진 고치기

```
edits <- bind_rows(
  Original = truck,
  Cropped  = truck |> filter(row > 40, col < 100),
  Flipped  = truck |> mutate(col = max(col) - col),
  Gray     = truck |> mutate(hex = gray((red + green + blue) / 3)),
  .id = "edit"
) |>
  mutate(edit = fct_inorder(edit))

edits |>
  ggplot(aes(col, row)) +
    geom_raster(aes(fill = hex)) +
    scale_fill_identity() +
    scale_y_reverse() +
    coord_equal() +
    facet_wrap(~edit, ncol = 2) +
    labs(title = "Editing pictures with dplyr", x = "Column", y = "Row")
```

자르기, 뒤집기, 흑백



- 자르기는 행 고르기다: `filter()`
- 뒤집기는 좌표 바꾸기다: `mutate()`. 7강의 대칭 변환이다.
- 흑백은 세 값의 평균이다: `mutate()`

사진 둘레의 띠

초록 테두리는 필름의 가장자리다.
이것도 픽셀이므로 **통계에 섞인다**.

1 사진을 표로

2 **색: RGB에서 HSV로**
색을 재는 다른 방법
컬렉션 훑기

3 사물 찾기: 컴퓨터 비전

4 사람 찾기: 얼굴과 자세

5 비슷한 사진 찾기: 임베딩

6 마무리

수 세 개로 만드는 색



왜 세 개인가

사람 눈의 원추세포가 세 종류이기 때문이다. 세 빛을 섞으면 우리가 보는 색 대부분을 만들 수 있어 화면도 세 빛만 낸다.

그래서 생기는 문제

`green` 값이 크다고 초록 사진이 아니다. `red`도 크면 노랑이고, 세 값이 모두 크면 흰색이다. R, G, B는 기계가 빛을 켜는 방식이지 사람이 색을 말하는 방식이 아니다.

HSV: 색을 말하는 방식

Value (명도)

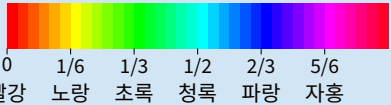
세 값 중 **가장 큰 값**. 0이면 검정이다.

Saturation (채도)

(최댓값 − 최솟값) / 최댓값. 0이면 회색, 1이면 순색이다. 분홍은 채도가 낮은 빨강이다.

Hue (색상)

무지개의 어디쯤인가. 0에서 1까지 돌아 다시 빨강으로 온다.



⚠ 교재 206쪽은 1/3을 파랑, 2/3을 초록이라 적었지만 반대다. 다음 슬라이드에서 직접 확인한다.

rgb2hsv(): 직접 확인

```r
# 순수한 초록
rgb2hsv(0, 1, 0,
        maxColorValue = 1)
```

```
      [,1]
h 0.3333333
s 1.0000000
v 1.0000000
```

	R, G, B	hue	sat	value
	1, 0, 0	0	1	1
	1, 1, 0	0.167	1	1
	0, 1, 0	0.333	1	1
	0, 1, 1	0.5	1	1
	0, 0, 1	0.667	1	1
	1, 0.75, 0.8	0.967	0.25	1
	0.5, 0.5, 0.5	0	0	0.5

💡 `hdir_pixels_long()`도 이 함수로 hue 등의 열을 만든다.

➔ 색상을 가진 것은 hue뿐이다. 밝기는 value, 진하기는 saturation으로 따로 잰다.

어두운 픽셀의 함정

```
1 pix |> select(row, col, red, green, blue, hue, saturation, value)
```

```
# A tibble: 15,750 × 8
    row   col   red green   blue   hue saturation value
  <dbl> <dbl> <dbl> <dbl>  <dbl> <dbl>      <dbl> <dbl>
1     1     1 0.122 0.204 0      0.234      1     0.204
2     2     1 0.145 0.224 0.0157 0.230      0.930 0.224
3     3     1 0.141 0.220 0.0275 0.235      0.875 0.220
# i 15,747 more rows
```

첫 픽셀의 채도가 1이다

눈에는 거의 검정인데 `blue`가 0이라 식으로는 "순색"이 된다. $(0.204 - 0)/0.204 = 1$.

⚠ 아주 어둡거나 탁한 픽셀의 `hue`와 `saturation`은 거의 아무 값이나 된다. 그래서 뒤에서 `value`와 `saturation`이 충분한 픽셀만 남기고 센다.

사진 열두 장의 색 분포

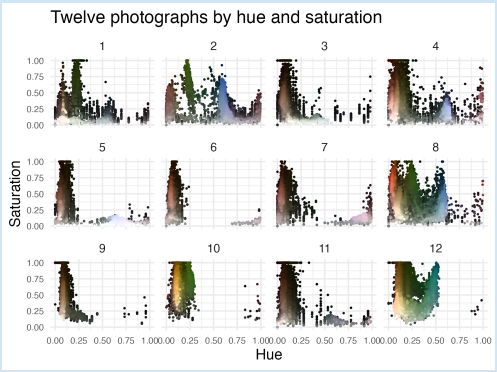
```
pix12 <- hdir_pixels_long(fsac$thm_path[1:12], fsac$filename[1:12])

pix12 |>
  mutate(photo = match(filename, fsac$filename)) |>
  arrange(photo, value) |>
  ggplot(aes(hue, saturation)) +
    geom_point(aes(color = hex), size = 0.3) +
    scale_color_identity() +
    facet_wrap(~photo, ncol = 4) +
    labs(
      title = "Twelve photographs by hue and saturation",
      x = "Hue", y = "Saturation"
    )
```

- `match()`는 표 `fsac`에서 몇 번째 사진인지를 알려 준다. 이 번호로 면을 나눈다.
- `arrange(photo, value)`: 나중에 찍는 점이 위에 오므로, 밝은 픽셀이 어두운 픽셀을 덮는다.

사진과 색 분포를 나란히





- 8번의 빨간 트럭은 왼쪽 끝, 하늘은 0.6 부근에 산을 이룬다.
- 12번의 청록색 하늘은 0.5 부근에 있다.
- 5번의 흰 하늘은 바닥에 깔린다 (채도가 낮다).

1,615장을 한꺼번에 펴면

```
# 실행하지 말 것 (교재 205쪽의 코드)
pix <- hdir_pixels_long(fsac$thm_path, fsac$filename)
```

```
# A tibble: 27,791,320 × 12
```

섬네일인데 2,779만 행

표의 크기만 2.3 GB, 만드는 동안에는 R이 6.5 GB까지 쓴다(강의자 컴퓨터에서 측정). Posit Cloud 무료 요금제의 메모리는 **1 GB**다. 세션이 죽는다.

이미지 자료의 생리

큰 사진은 섬네일의 약 46배다(1024×715 vs. 150×105). 이미지 자료는 금세 컴퓨터보다 커진다.

➔ 사실 픽셀 전부가 필요하지 않다. 필요한 것은 **사진마다 수 몇 개**다. 그러니 한 장씩 요약하고 픽셀은 버린다.

한 장씩 요약하고 버린다

```
color_one <- function(path, name) {
  hdir_pixels_long(path, name) |>
    group_by(filename) |>
    summarize(
      avg_value = mean(value),
      avg_green = mean(green),
      sat_large = mean(saturation > 0.8 & value > 0.5)
    )
}
color <- map2(fsac$thm_path, fsac$filename, color_one) |>
  list_rbind() |>
  inner_join(fsac, by = "filename")
```

- 함수를 만드는 법은 교재 11장의 내용이다. 이 틀에서 `summarize()` 안만 바꾸어 쓴다.
- `map2()`가 한 쌍씩 함수에 넣고 `list_rbind()`가 쌓는다. 강의자 컴퓨터에서 약 10초.

가장 어두운 사진

```
color |> arrange(avg_value) |> select(avg_value, title)
```

```
# A tibble: 1,615 × 2
  avg_value title
      <dbl> <chr>
1    0.0614 Illinois Central R.R., Chicago, Ill. Vernon Brower, riding the …
2    0.0700 Switch lights in Santa Fe R.R. yards, Los Angeles, Calif. Due t…
3    0.0710 Daniel Anastazia, blacksmith's helper at the roundhouse of the …
4    0.0765 Daniel Anastazia, blacksmith's helper, Rock Island R.R., Blue I…
5    0.0804 Night view of part of the South Water street freight terminal o…
# i 1,610 more rows
```

```
dark <- color |>
  filter(height < width) |>
  arrange(avg_value) |>
  slice(1:14)
img <- hdir_plot_images(dark$thm_path, ncol = 7)
writeJPEG(img, "dark.jpg")
```

💡 `hdir_plot_images()`는 사진들을 격자 한 장으로 붙인다. 가로 사진만 골라야 칸이 고르다.

어두운 사진: 밤의 철도, 공장의 내부



- 제목으로도 그림으로도 야간의 철도, 기관차 정비소, 비행기 공장, 전차다.
- 수 한 개로 줄 세웠을 뿐인데 컬렉션 안의 한 **무리**가 드러난다. 검은 배경에 기계만 빛나는, 전시 생산의 사진들이다.
  - 둘째 사진의 제목에는 **등화관제**(blackout regulations)가 나온다. 그런데 표의 연도는...

메타데이터도 자료다: year

```
1 fsac |> count(year)
```

```
# A tibble: 3 × 2
   year     n
  <dbl> <int>
1  1939  1598
2  1941     3
3  1942    14
```

대조해 보니

filename은 미국 의회도서관의 항목 번호다. 2017878660(해병대 글라이더)을 찾으면 **1942년 5월**인데 표에는 1939다. 컬렉션 안내도 촬영 시기를 1939–1944년이라 적는다.

- 교재(209쪽)는 이 열을 따라 "1939년 여름"이라 적었다. 이 열로는 시기별 비교를 할 수 없다.
- → 픽셀만 의심할 일이 아니다. 왜 이렇게 되었는지 모를 때는 원 기록(https://www.loc.gov/item/2017878660)으로 돌아간다.

green 평균이 가장 높은 사진

```
color |> arrange(desc(avg_green)) |> select(avg_green, title)
```

```
# A tibble: 1,615 × 2
  avg_green title
      <dbl> <chr>
1     0.609 In the flat cattle country in the south western tip of Puerto R…
2     0.603 Chopping cotton on rented land near White Plains, Greene County…
3     0.594 North American's P-51 Mustang Fighter is in service with Britain…
# i 1,612 more rows
```



➔ 초록이 아니라 **흰 구름과 밝은 하늘**이다. 앞에서 말한 함정이 실제로 일어났다.

채도: 진하고 밝은 픽셀의 비율

```
color |> arrange(desc(sat_large)) |> select(sat_large, title)
# A tibble: 1,615 × 2
  sat_large title
      <dbl> <chr>
1     0.304 "Marine with the training gliders at Page Field, Parris Island,…
2     0.245 "Marine with the gliders at Page Field, Parris Island, S.C."
3     0.239 "A Marine Corps glider being towed by a power plane from Page F…
# i 1,612 more rows
```



➔ 새파란 하늘과 노란 글라이더, 오렌지. `saturation > 0.8 & value > 0.5`는 어두운 픽셀의 함정을 피한 조건이다.

색상은 평균을 내면 안 된다

색상은 고리다

0.01과 0.99는 두 값 모두 빨강인데, 평균 내면 0.5, 곧 청록이 된다.



구간으로 나누어 센다

`floor(hue * 12)`: 12를 곱해 내림하면 0–11의 구간 번호(hb)가 된다. 그리고 사진마다 구간별 픽셀의 비율을 잰다.

⚠ 12칸으로 나누는 것도, 색의 경계를 정하는 것도 **우리의 선택**이다. 빨강이 어디서 끝나는지는 자료에 적혀 있지 않다.

같은 틀에 조건만 바꾸어

```
hue_one <- function(path, name) {
  hdir_pixels_long(path, name) |>
    filter(value > 0.2, saturation > 0.2) |>
    mutate(hb = floor(hue * 12)) |>
    group_by(hb, filename) |>
    summarize(prop = n() / first(nrow * ncol) * 100, .groups = "drop")
}

land <- fsac |> filter(height < width)
hues <- map2(land$thm_path, land$filename, hue_one) |> list_rbind()

top_hue <- hues |>
  group_by(hb) |>
  arrange(desc(prop)) |>
  slice(1:7) |>
  ungroup() |>
  inner_join(fsac, by = "filename")
```

구간마다 일곱 장 (1): 따뜻한 쪽

```
1 img <- hdir_plot_images(filter(top_hue, hb < 6)$thm_path, ncol = 7)
```



- 0: 벽돌집, 용광로, 밀밭
- 1: 하늘에서 본 밭, 흙
- 2: 풀과 작물
- 3–5: 들판, 그리고 청록빛 하늘

⚠ 구간 3에는 **초록 테두리**를 두른 사진이 많다. 찍힌 대상이 아니라 필름의 띠가 순위를 올렸다.

구간마다 일곱 장 (2): 차가운 쪽

```
1 img <- hdir_plot_images(filter(top_hue, hb >= 6)$thm_path, ncol = 7)
```



- 6–7: 하늘과 비행기, 성조기
- 8–9: 방공 기구, 보랏빛 하늘
- 10–11: 분홍빛 실내

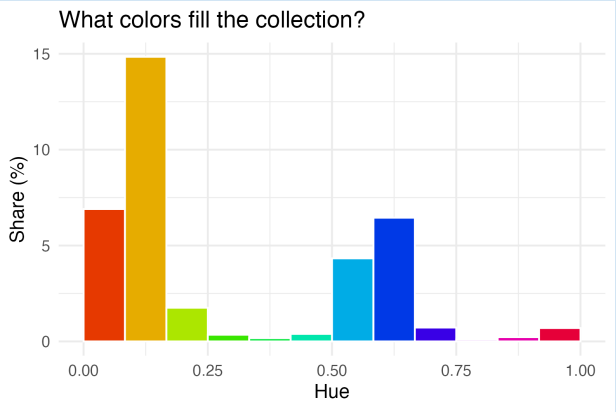
⚠ 분홍빛 무도회는 분홍 방이 아니다. 교재는 세월에 필름이 **변색**한 것으로 본다. 색은 찍힌 세계와 필름의 역사를 함께 담는다.

컬렉션 전체는 어떤 색인가

```
hues |>
  group_by(hb) |>
  summarize(share = sum(prop) / nrow(land)) |>
  mutate(
    mid = (hb + 0.5) / 12,
    color = hsv(mid, 1, 0.9)
  ) |>
  ggplot(aes(mid, share)) +
    geom_col(aes(fill = color), width = 1 / 12, color = "white") +
    scale_fill_identity() +
    labs(
      title = "What colors fill the collection?",
      x = "Hue", y = "Share (%)"
    )
```

- share: 사진 면적의 평균 몇 %가 그 구간인가. 구간이 없는 사진은 행이 없어서 합을 사진 수로 나눈다.

주황과 파랑의 컬렉션



- ➔ 주황·노랑과 파랑이 대부분이다. 순수한 초록은 거의 없다.
- ➔ 다 합해도 면적의 37%다. 나머지는 어둡거나 탁하다.
- ⚠ 1940년대 세상의 색인가, 필름의 색인가? 픽셀만으로는 가를 수 없다.

1 사진을 표로

2 색: RGB에서 HSV로

3 사물 찾기: 컴퓨터 비전
픽셀에서 의미로
객체 탐지

4 사람 찾기: 얼굴과 자세

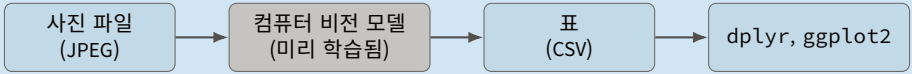
5 비슷한 사진 찾기: 임베딩

6 마무리

픽셀로는 물을 수 없는 것

색은 세었다. 그런데

"사람이 몇 명인가?", "비행기가 있는가?", "이 사진과 비슷한 사진은?"은 픽셀을 아무리 묶어도 답이 나오지 않는다.



컴퓨터 비전(computer vision)

사람의 시각을 자료와 코드로 흉내 내는 분야. 사진을 넣으면 "무엇이 어디에 있다"는 **구조화된 표**를 내놓는다.

➔ 8강의 `cleanNLP`와 같은 그림이다. 문장을 넣으면 표가 나왔던 것처럼, 사진을 넣으면 표가 나온다.

모델은 이미 돌려 두었다

```
# 실행하지 않는다 (Python 환경이 필요하다)
hdir_init_pipeline(annotations = c("object", "face", "pose", "embed"))
hdir_apply_pipeline(fsac$med_path, fsac$filename, outdir = "data")
```

주석	찾는 것	결과 파일 (hdr/data/)
object	사물의 종류와 상자	fsac_nn_inst.csv.bz2
face	얼굴의 상자	fsac_nn_face.csv.bz2
pose	몸의 관절 17곳	fsac_nn_kpnt.csv.bz2
embed	사진을 나타내는 수	fsac_embd_pca, _umap, fsac_knn

- 교재 저자가 큰 사진 1,600여 장에 돌린 결과다. 노트북으로 약 1시간 걸린다(교재). 우리는 결과만 읽는다.

➔ 그러니 오늘의 기술은 모델을 만드는 것이 아니라 **모델의 표를 읽고 의심하는 것**이다.

텍스트와 무엇이 다른가

텍스트의 주석 (8강)

단어와 문장이라는 약속된 기호가 이미 있다. 토큰을 이어 붙이면 원문이 되살아난다. 주석은 정보를 **더하기만** 한다.

이미지의 주석

사진은 대상을 닮아서 뜻을 낸다. 나눌 약속된 기호가 없다. 표에서 사진을 되살릴 수 없다. 모델은 정보를 만들면서 **버린다**.

그래서

어느 표도 사진의 뜻을 다 담지 못한다. 어떤 주석을 쓰고 어떻게 겹칠지는 언제나 **해석적 선택**이다(교재 10.4절).

➔ 시작 전에 정해 두자. 표의 값은 사진에 대한 사실이 아니라 **한 모델의 판독**이다.

객체 탐지의 표

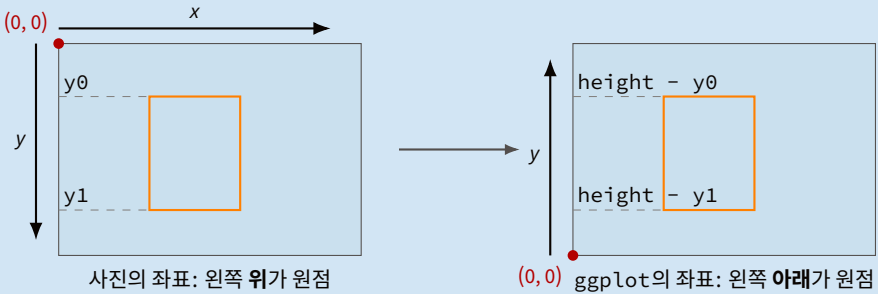
```
inst <- read_csv(file.path("hdr", "data", "fsac_nn_inst.csv.bz2"))
inst |> select(filename, class, prob, x0, x1, y0, y1)
```

```
# A tibble: 6,789 × 7
    filename class   prob    x0    x1    y0    y1
       <dbl> <chr>  <dbl> <dbl> <dbl> <dbl> <dbl>
1 2017877340 person 0.998  268.  335.  170.  428.
2 2017877340 person 0.997  377.  456.  176.  423.
3 2017877340 horse  0.995  695.  923.  305.  593.
4 2017877340 horse  0.881  573.  761.  331.  588.
# i 6,785 more rows
```

- 한 행이 찾아낸 사물 한 개다. 첫 사진에서 사람 두 명, 말 두 마리를 찾았다. `filename`으로 `fsac`과 잇는다.
- `class`: 사물의 종류. 정해진 어휘 안에서만 고른다(이 표에는 76가지).
- `x0`–`y1`: 사물을 두르는 상자(bounding box)의 네 변. 픽셀 단위다.

⚠ `prob`는 이름과 달리 **확률이 아니라 신뢰도 점수**다. 줄 세우는 데에만 믿는다.

상자의 y는 위에서부터



➔ 앞에서 `scale_y_reverse()`로 뒤집었던 바로 그 문제다.

- 상자의 위 변은 `height - y0`, 아래 변은 `height - y1`이다. 이번에는 사진을 깔아야 하므로 값을 직접 빼서 뒤집는다.

사진 위에 상자 그리기 (1): 표 준비

```
fsac_one <- filter(fsac, filename == 2017877340)
boxes <- inst |>
  semi_join(fsac_one, by = "filename") |>
  mutate(
    label = sprintf("%s (%d%%)", class, round(prob * 100)),
    top = height - y0,
    bottom = height - y1
  )
boxes |> select(class, x0, x1, top, bottom, label)
```

```
# A tibble: 4 × 6
  class     x0    x1   top bottom label
  <chr>  <dbl> <dbl> <dbl>  <dbl> <chr>
1 person  268.  335.  546.   288. person (100%)
2 person  377.  456.  540.   293. person (100%)
3 horse   695.  923.  411.   123. horse (100%)
4 horse   573.  761.  385.   128. horse (88%)
```

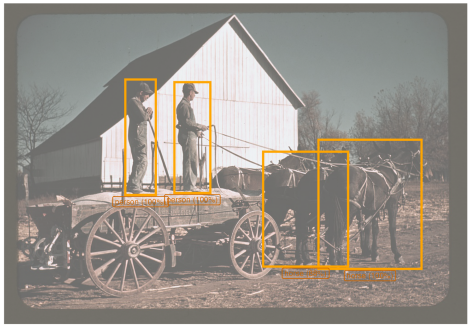
💡 `sprintf()`: 틀에 값을 넣어 글자를 만든다(%s 글자, %d 정수, %%는 % 자체).

사진 위에 상자 그리기 (2): 층 쌓기

```
ggplot(boxes) +
  geom_rect_img(                    # 사진을 맨 먼저 깐다 (ggimg)
    aes(img = med_path, xmax = width, ymax = height),
    xmin = 0, ymin = 0, alpha = 0.6, data = fsac_one
  ) +
  geom_rect(
    aes(xmin = x0, xmax = x1, ymin = bottom, ymax = top),
    color = "orange", fill = NA, linewidth = 1
  ) +
  geom_label(
    aes(x = (x0 + x1) / 2, y = bottom - 15, label = label),
    size = 2.6, color = "darkorange3"
  ) +
  scale_x_continuous(limits = c(1L, fsac_one$width)) +
  scale_y_continuous(limits = c(1L, fsac_one$height)) +
  labs(title = "Objects found in one photograph") +
  theme_void()
```

모델이 본 것과 보지 못한 것

Objects found in one photograph



본 것

사람 두 명과 말 두 마리의 자리가 정확하다.

보지 못한 것

겹쳐 선 왼쪽 뒤의 말에는 상자가 없다. 헛간, 수레, 나무도 없다. **어휘에 없기** 때문이다.

큰 사진은 2017877340과 2017877351 두 장뿐이다. 다른 사진은 `img = thm_path` 로 바꾸면 된다.

컬렉션 전체에서 세기

```
inst |>
  filter(prob > 0.9) |>
  group_by(class) |>
  summarize(count = n()) |>
  arrange(desc(count))
```

```
# A tibble: 46 × 2
  class    count
  <chr>    <int>
1 person    2560
2 airplane   123
3 horse       72
4 car         71
5 train       67
# i 41 more rows
```

```
inst |>
  summarize(
    all = n(),
    over_07 = sum(prob > 0.7),
    over_09 = sum(prob > 0.9)
  )
```

```
# A tibble: 1 × 3
    all over_07 over_09
  <int>   <int>   <int>
1  6789    4907    3147
```

⚠ 문턱은 우리가 정한다. 0.9면 절반이 넘게 사라진다.

➔ 사람, 탈것, 가축. 표로도 사람과 교통과 농장의 컬렉션이 보인다.

신뢰도 0.9가 넘는 연 12개

```
inst |>
  filter(class == "kite", prob > 0.9) |>
  inner_join(fsac, by = "filename") |>
  select(prob, title)
```



- 사진 11장(상자 12개)에 **연은 없다**. 성조기 3장, 방공 기구 6장, 낙하산 1장, 협곡 1장이다.
- → 어휘에 방공 기구가 없으니 아는 말 중 가장 닮은 것을 불렀다. 점수가 높다고 맞는 것이 아니다.

1 사진을 표로

2 색: RGB에서 HSV로

3 사물 찾기: 컴퓨터 비전

4 **사람 찾기: 얼굴과 자세**
얼굴
자세

5 비슷한 사진 찾기: 임베딩

6 마무리

얼굴 탐지의 표

```
face <- read_csv(file.path("hdr", "data", "fsac_nn_face.csv.bz2"))
face |> select(filename, x0, x1, y0, y1, prob, height, width)
```

```
# A tibble: 1,428 × 8
    filename     x0    x1    y0    y1  prob height width
       <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>  <dbl> <dbl>
1 2017877340   727   751   591   620 0.904    716  1024
2 2017877340   305   320   198   219 0.794    716  1024
3 2017877350   416   461   498   554 0.936    729  1024
# i 1,425 more rows
```

탐지(detection) vs. 식별(recognition)

탐지는 “여기에 얼굴이 있다”이고, 식별은 “이 얼굴은 누구다”이다. 오늘은 탐지만 한다. 식별은 이 컬렉션에서는 쓸모가 적고(교재), 살아 있는 사람의 사진에서는 윤리와 법의 문제가 된다.

➔ 객체 표와 같은 모양이다. 모든 행이 얼굴이므로 `class`만 없다. 코드도 거의 그대로 쓴다.

같은 코드, 다른 사진

```
fsac_two <- filter(fsac, filename == 2017877351)
face |>
  semi_join(fsac_two, by = "filename") |>
  ggplot() +
    geom_rect_img(
      aes(img = med_path, xmax = width, ymax = height),
      xmin = 0, ymin = 0, alpha = 0.6, data = fsac_two
    ) +
    geom_rect(
      aes(xmin = x0, xmax = x1,
          ymin = height - y1, ymax = height - y0),
      color = "orange", fill = NA, linewidth = 1
    ) +
    scale_x_continuous(limits = c(1L, fsac_two$width)) +
    scale_y_continuous(limits = c(1L, fsac_two$height)) +
    labs(title = "Faces found in one photograph") +
    theme_void()
```

여덟 명 중 여섯 명

Faces found in one photograph



- 얼굴이 아주 작은데도 여섯 명을 찾았다.
- 놓친 두 명은 고개를 돌렸거나 모자 그늘에 얼굴이 묻혔다.
- 앞의 객체 탐지는 같은 사진에서 여덟 명을 다 찾았다.

➔ 얼굴의 개수가 곧 사람의 수는 아니다. 어느 표로 사람을 셀지 정하는 것도 선택이다.

사진마다 얼굴이 몇 개인가

```
n_face <- face |>
  filter(prob > 0.9) |>
  count(filename, name = "n_face")

fsac |>
  left_join(n_face, by = "filename") |>
  mutate(n_face = replace_na(n_face, 0)) |>
  count(n_face)
```

```
# A tibble: 17 × 2
  n_face     n
   <int> <int>
1      0  1059
2      1   362
3      2    99
4      3    38
5      4    14
# i 12 more rows
```

⚠ **표에 없는 사진이 얼굴 0개다.** face만 세면 556장만 나오고 0이 사라진다. 그래서 fsac에서 출발해 left_join으로 잇는다(5강).

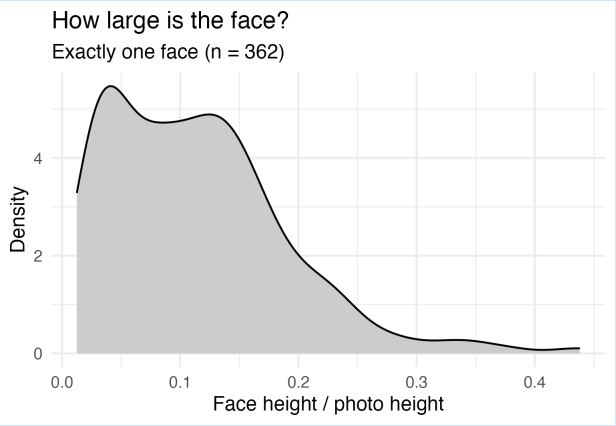
➔ 전체의 66%에는 믿을 만한 얼굴이 없고, 얼굴 하나인 사진이 362장(22%)이다. 가장 많은 사진은 74개(폭격기 공장의 노동자들)다.

얼굴의 크기: 사진의 거리

```
face |>
  filter(prob > 0.9) |>
  semi_join(filter(n_face, n_face == 1), by = "filename") |>
  mutate(prop = (y1 - y0) / height) |>
  ggplot() +
    geom_density(aes(x = prop), fill = "grey80") +
    labs(
      title = "How large is the face?",
      subtitle = "Exactly one face (n = 362)",
      x = "Face height / photo height",
      y = "Density"
    )
```

- 사진 높이에서 얼굴 높이가 차지하는 비율이다. 얼굴이 하나인 사진만 골라야 뜻이 분명하다.
- ⚠ 교재(221쪽)는 prob 필터 없이 이어서 사진 362장의 얼굴 425개를 그린다. 먼저 걸러 362개로 그렸다.

사람을 얼마나 멀리서 찍었나



- 4% 부근과 12% 부근에 봉우리가 있고, 절반 가까이가 10% 아래다.
- 가장 큰 얼굴은 사진 높이의 44%다.

수로 옮긴 샷

영화의 원경·중경·근접 구분과 통한다. 인물이 있어도 대개 멀리서 찍었다.

자세 탐지: 몸의 17곳

```
kpnt <- read_csv(file.path(
  "hdr", "data", "fsac_nn_kpnt.csv.bz2"
))
kpnt |>
  select(index, kpname, x, y, score)
```

```
# A tibble: 84,524 × 5
  index kpname        x     y score
  <dbl> <chr>     <dbl> <dbl> <dbl>
1     0 nose       318.  210. 0.289
2     0 left_eye   321.  205. 0.257
3     0 right_eye  316.  204. 0.249
4     0 left_ear   303.  201. 0.214
# i 84,520 more rows
```

상자가 아니라 점

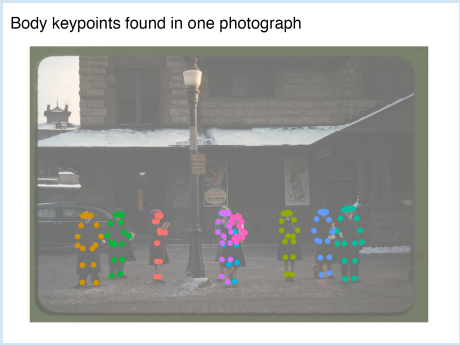
코·눈·귀, 어깨·팔꿈치·손목, 엉덩이·무릎·발목. 한 행이 점 한 개이고, 같은 index의 17개가 한 사람이다.

⚠ 찾은 사람마다 17개를 **언제나** 내놓는다. 가려져 안 보이는 곳도 어림해 찍고, score만 낮게 준다.

💡 초상 사진이 많은 컬렉션이라면 몸의 방향이나 시선을 읽는 데에도 쓸 수 있다.

점을 사진 위에

```
# 나머지 층은 얼굴 그림과 같다
geom_point(
  aes(
    x = x,
    y = height - y,
    color = factor(index)
  ),
  show.legend = FALSE
)
```



⚠ 왼쪽 세 명, 오른쪽 세 명은 잘 잡았다. 그러나 8명인데 index는 9개다. 겹쳐 선 가운데 두 명을 세 명으로 쪼갰다.

➔ 얼굴을 놓친 두 사람은 여기서 잡혔다. 모델마다 실패하는 자리가 다르다.

모델이 몸의 어디를 믿는가

```
kpnt |>
  group_by(kpname) |>
  summarize(mu = mean(score),
            med = median(score)) |>
  arrange(desc(mu))
```

```
# A tibble: 17 × 3
  kpname          mu    med
  <chr>        <dbl>  <dbl>
1 left_eye    0.331  0.122
2 right_eye   0.329  0.119
3 nose        0.305  0.121
4 right_ear   0.198  0.0946
5 left_ear    0.194  0.0971
6 right_elbow 0.0957 0.0445
# i 11 more rows
```



➔ 점이 클수록 평균 점수가 높다.

➔ 모델의 한계이자 **컬렉션의 성격**이다. 기계나 사람에 가린 하체, 작게 찍힌 사람들. 실패도 자료다.

1 사진을 표로

2 색: RGB에서 HSV로

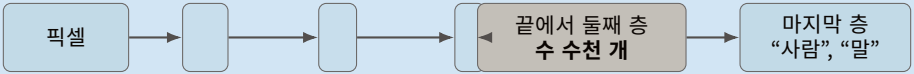
3 사물 찾기: 컴퓨터 비전

4 사람 찾기: 얼굴과 자세

5 **비슷한 사진 찾기: 임베딩**
사진의 지도
최근접 이웃

6 마무리

임베딩: 사진을 나타내는 수



딥러닝(deep learning)과 전이 학습

픽셀을 여러 단계로 조금씩 바꾸어 이름표에 이른다. 다 배운 모델에서 **마지막 단계만 떼어내면** 남은 수들이 사진을 요약한 벡터가 된다. 이것이 **임베딩**이다.

- 수 하나하나에는 뜻이 없다. 그러나 비슷한 사진은 가까운 벡터를 가진다. 사물뿐 아니라 구도까지 함께 묻어 온다.

→ 7강의 벡터와 거리, 8강의 문서 벡터와 같다. 사진을 수의 나열로 놓으면 거리를 잴 수 있다.

- 교재는 4,096개라 적었고, 지금의 `hdir` 코드는 1,280개를 만든다. 우리는 줄인 결과만 받았다.

PCA로 사진 늘어놓기

```
pca <- read_csv(file.path("hdr", "data", "fsac_embd_pca.csv.bz2"))
pca |> select(filename, pca_01, pca_02, pca_03)
```

```
# A tibble: 1,615 × 4
    filename pca_01 pca_02 pca_03
       <dbl>  <dbl>  <dbl>  <dbl>
1 2017877340   26.9   2.88  -8.42
2 2017877341   26.0  -4.02  -3.02
# i 1,613 more rows
```

```
set.seed(1)
pca |>
  slice_sample(n = 250) |>
  inner_join(fsac, by = "filename") |>
  ggplot(aes(pca_01, pca_02)) +
    geom_point_img(aes(img = thm_path), size = 1) +
    labs(title = "Photographs placed by PCA of their embeddings") +
    theme_void()
```

- 좌표 25개가 들어 있다. `geom_point_img()`는 점 대신 사진을 찍는다.

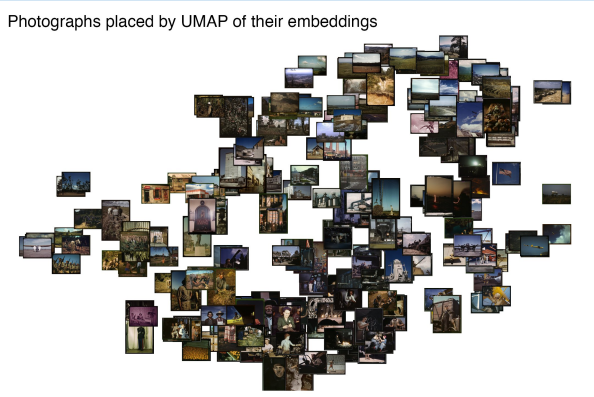
PCA 지도



- **위**: 사람이 있는 사진, 실내
- **아래**: 풍경
  - 위아래라는 큰 흐름은 보이지만 가운데는 빽빽하다.

💡 축 이름은 달지 않았다. `pca_01`의 뜻은 모델이 알려 주지 않는다(8강). 읽어 내는 것은 사람이다.

UMAP 지도



- 위 오른쪽: 풍경
- 오른쪽: 비행기, 그 위에 밤 사진
- 아래 가운데: 일하는 사람의 인물 사진
- 왼쪽: 밭과 농사

⚠ `fsac_embd_umap.csv.bz2`로 같은 코드를 돌렸다. UMAP은 **가까운 것**만 지키므로(8강) 무리 사이의 거리는 믿지 않는다.

사진마다 가장 가까운 50장

```
knn <- read_csv(file.path("hdr", "data", "fsac_knn.csv.bz2"))
knn |> select(filename, rank, filename_n, distance)
```

```
# A tibble: 80,750 × 4
    filename  rank filename_n distance
       <dbl> <dbl>      <dbl>    <dbl>
1 2017877340     1 2017877500     15.1
2 2017877340     2 2017877495     16.1
3 2017877340     3 2017877501     16.9
# i 80,747 more rows
```

- $1{,}615 \times 50 = 80{,}750$행. `distance`는 **작을수록 가깝다**. `rank`는 그 순위다.

⚠ 교재(229쪽)는 UMAP에 기반한 이웃이라 했지만 거리를 직접 재 보면 PCA 좌표 25개의 거리와 같다(15.12). UMAP으로 재면 0.27이다.

➔ 쓰임: 공공 아카이브의 “비슷한 사진” 추천(Photogrammar). 9강처럼 에지로 삼으면 사진의 네트워크가 된다.

이웃을 격자로 (1): 시작 사진

```
land <- fsac |> filter(height < width)
start <- land |>
  inner_join(knn, by = "filename") |>
  filter(rank == 1) |>
  arrange(distance) |>
  slice(seq(1, 11, by = 2))

self <- knn |>
  filter(rank == 1) |>
  mutate(filename_n = filename, rank = 0, distance = 0)
```

- `start`: 가장 가까운 이웃과의 거리가 짧은 사진부터 고른다. 가까운 쌍은 서로의 이웃으로 두 번 나오므로 `seq(1, 11, by = 2)`로 한 장씩만 집어 여섯 장을 남긴다.
- `self`: 표에는 자기 자신으로 가는 행이 없다. 첫 칸에 시작 사진을 놓으려고 거리 0인 행을 만든다.

이웃을 격자로 (2): 일곱 장씩

```
nn <- knn |>
  bind_rows(self) |>
  semi_join(start, by = "filename") |>
  inner_join(fsac, by = c("filename_n" = "filename")) |>
  filter(height < width) |>
  group_by(filename) |>
  arrange(filename, distance) |>
  slice(1:7)

img <- hdir_plot_images(nn$thm_path, ncol = 7)
writeJPEG(img, "nn.jpg")
```

- 조인 키의 이름이 다르면 `by = c("filename_n" = "filename")`으로 짝을 알려 준다(5강). 이웃 사진의 경로를 얻는 조인이다.
- `ncol = 7`이 `slice(1:7)`과 맞으므로 시작 사진이 한 줄씩 나온다.

오류 없이 틀린 코드: 교재 230쪽

```r
# 교재
arrange(filename, desc(distance))
```

```
    filename  rank distance
1 2017877448    50     12.8
2 2017877448    49     12.8
3 2017877448    48     12.8
4 2017877448    47     12.8
```

```r
# 고친 코드
arrange(filename, distance)
```

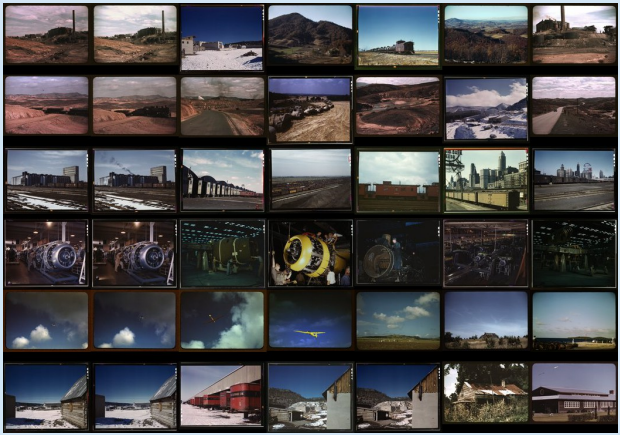
```
    filename  rank distance
1 2017877448     0     0
2 2017877448     1     4.08
3 2017877448     2     8.42
4 2017877448     3     8.66
```

한 글자가 뜻을 뒤집었다

`desc()`가 있으면 50장 중 **가장 먼** 일곱 장을 고른다. 시작 사진도 빠진다. 오류는 나지 않고 그림도 그럴듯하다.

➔ 출력 몇 줄을 찍어 `rank`와 `distance`를 눈으로 보면 바로 알 수 있다. 출판된 교재의 코드도 예외가 아니다.

첫 열이 시작 사진, 오른쪽으로 갈수록 멀다



- 1–2행: 구리 광산의 민둥산
- 3행: 곡물 창고, 철도 조차장
- 4행: 공장 안의 큰 기계
- 5행: 구름과 하늘
- 6행: 눈밭과 건물

➔ 1행에는 분홍 하늘과 파란 하늘이 섞여 있다. 색이 아니라 **구도와 내용**이 묶었다.

➔ 맨 앞 두 칸은 거의 같은 사진이다. 사진가가 같은 자리에서 여러 번 찍었다.

1 사진을 표로

2 색: RGB에서 HSV로

3 사물 찾기: 컴퓨터 비전

4 사람 찾기: 얼굴과 자세

5 비슷한 사진 찾기: 임베딩

6 **마무리**

사진 표 앞에서 물을 것

표를 믿기 전에

1. 이 값은 찍힌 세계의 것인가, 필름의 것인가, 스캔의 것인가? (변색, 테두리)
2. 모델의 어휘에 내 질문의 대상이 있는가? (방공 기구, 헛간)
3. 높은 점수의 결과를 사진으로 직접 열어 보았는가? (연)
4. 표에 없는 사진은 무엇인가? (얼굴 0개)
5. 메타데이터는 믿을 만한가? (`year`)
6. 문턱과 구간은 누가 정했는가? (0.9, 12칸)

수천 장을 멀리서 보는 방법이지만, **몇 장은 반드시 가까이서** 본다. 멀리 보기와 가까이 보기는 번갈아 간다.

모든 컬렉션을 자유롭게 쓸 수 있는 것은 아니다. 분석하고 싣기 전에 저작권과 이용 조건을 확인한다.

오늘의 함수

함수	하는 일	패키지
`gimg_load_img()`	사진 파일을 배열로 읽기	ggimg
`hdir_pixels_long()`	사진을 픽셀 한 행의 표로 펴기	hdir
`hdir_plot_images()`·`writeJPEG()`	사진 격자 만들기, 저장하기	hdir, jpeg
`geom_raster()`·`scale_fill_identity()`	격자 칠하기, 값을 색으로 그대로	ggplot2
`rgb2hsv()`·`hsv()`	RGB를 HSV로, HSV를 색으로	grDevices
`map2()`·`list_rbind()`	한 쌍씩 함수에 넣기, 결과 쌓기	purrr
`geom_rect_img()`·`geom_point_img()`	사각형에 사진 깔기, 점 대신 사진	ggimg
`geom_rect()`·`geom_label()`	상자와 이름표 그리기	ggplot2
`sprintf()`	틀에 값을 넣어 글자 만들기	base
`hdir_init_pipeline()`	모델 준비 (Python 필요)	hdir
`hdir_apply_pipeline()`	모델 돌려 표 만들기	hdir

오늘 배운 것

픽셀

- 사진은 행 $\times$ 열 $\times$ 채널의 배열
- 표로 펴면 `dplyr`, `ggplot2` 그대로
- 한 장씩 요약하고 버린다

탐지

- 모델의 판독이 표로 온다
- 사진의 y는 위에서부터
- 어휘와 문턱, 점수를 의심한다

색

- RGB의 함정, HSV로 재기
- 색상은 평균 말고 구간으로

임베딩

- 사진을 나타내는 수의 벡터
- PCA·UMAP의 지도, 최근접 이웃

확인 질문

스스로 답해 보기

1. `dim(img)`가 `105 150 3`이다. 세 수는 각각 무엇이고, 픽셀 표는 몇 행이 되는가?
2. `green` 평균이 높은 사진이 초록 사진이 아닌 까닭은? 초록 사진을 찾으려면 어떻게 하는가?
3. 색상 0.01과 0.99를 평균 내면 왜 안 되는가?
4. 사진 위에 상자를 그릴 때 `height - y0`처럼 빼는 까닭은?
5. "신뢰도 0.9 이상의 연이 12개이므로 이 컬렉션에 연날리기 사진이 있다"는 주장은 왜 틀렸는가?
6. 사진마다 얼굴 수를 셀 때 `fsac`에서 출발해 `left_join`하는 까닭은?
7. 교재의 `arrange(filename, desc(distance))`는 어떤 사진을 고르는가?

답이 막히는 항목이 있으면 eTL Q&A 게시판에 질문을 올린다 (참여 점수 1점).

다음 시간

- 16강 (2026-11-11 수) — 이미지 데이터 사례 연습
  - 웹툰 섬네일 논문을 읽고, 오늘의 픽셀 표로 섬네일의 특징을 직접 잰다.
  - 잰 값을 메타데이터와 이어 논문의 결과와 비교한다.
- 기말고사 (2026-11-16 월) — 범위: 9–16강

더 보고 싶다면

- Arnold & Tilton, *Distant Viewing: Computational Exploration of Digital Images* (MIT Press, 2023)
- Manovich, *Cultural Analytics* (MIT Press, 2020)
- FSA-OWI 컬러 사진 (미국 의회도서관) — `https://www.loc.gov/collections/fsa-owi-color-photographs/`
- Photogrammar — `https://photogrammar.org/`